team08 1 주차 보고서

항목	내용
팀 번호	08 (개인)
이름	신민지
학번	2024245145
GitHub ID	godmj
저장소	RobotAlgorithm-GPS-Localization-team08
제출일	2026 년 11 월 07 일

1. 팀 Git Repository 만들기

안내문의 이름과 폴더 구조대로 저장소를 만들고, 자기소개 파일을 push 한 뒤 GitHub 페이지와 ZIP 다운로드로 업로드를 확인했다.

1.0 작업 환경

항목	내용
OS	Windows 11 Home
Python	3.13
Git	Git for Windows (MSYS2 UCRT64 터미널에서 사용)
에디터	VS Code
Git 사용자 설정	user.name = godmj, user.email = GitHub 계정 이메일

1.1 프로젝트 저장소 만들기

GitHub 에서 안내문에 지정된 이름 RobotAlgorithm-GPS-Localization-team08 로 새 저장소를 만들었다. 조교님이 저장소 주소로 확인할 수 있도록 Public 으로 설정하고, README 와 Python 용 .gitignore 를 함께 생성했다. 개인으로 진행하므로 팀원 초대는 해당하지 않는다.

1.2 폴더 구조 만들기

저장소를 내 컴퓨터로 내려받아(clone) 안내문의 폴더 구조대로 만들고, team.md 에 팀 번호·이름·학번·GitHub ID 를 적었다. Git 은 빈 폴더를 업로드하지 않아 빈 폴더에는 .gitkeep 파일을 넣었다.

1.3 자기소개 파일 작성 및 push

2024245145_신민지.md 에 300 자 이내(270 자) 자기소개를 작성하고 git add → commit → push 로 올렸다.

1.4 업로드 확인

GitHub 저장소 페이지에서 team.md, 자기소개 파일, src, week1 폴더가 올라간 것을 확인했다. 또한 Code → Download ZIP 으로 저장소를 내려받아 압축을 풀고, 한글 파일명(2024245145_신민지.md)이 깨지지 않고 내용도 정상적으로 열리는 것을 확인했다.

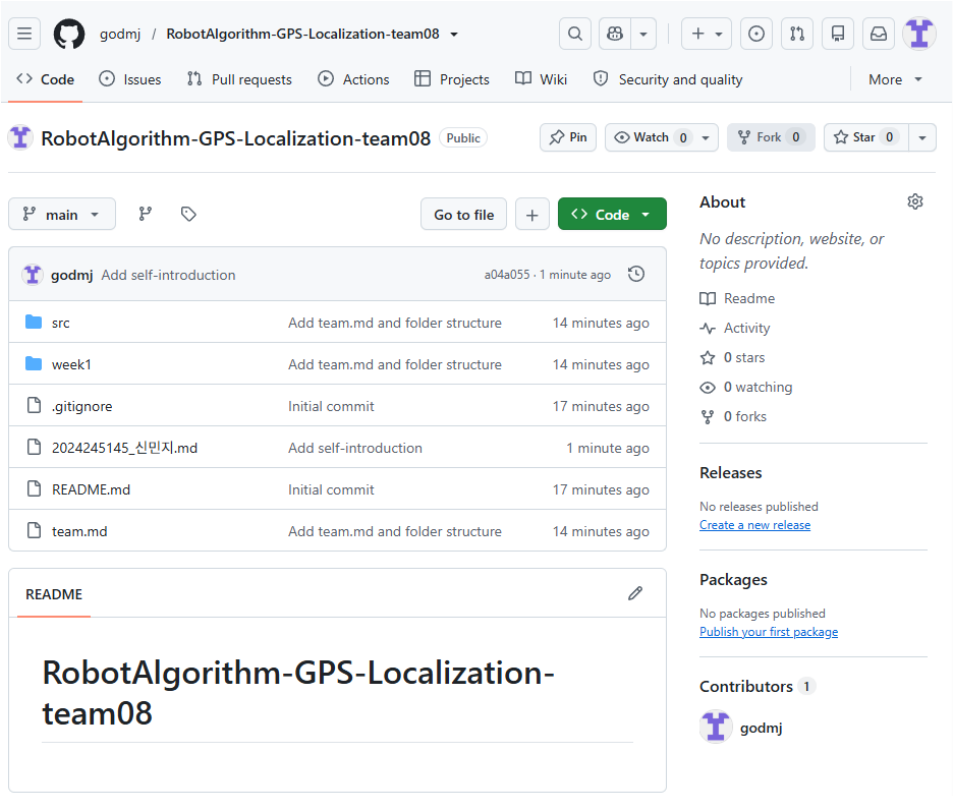


그림 1. GitHub 저장소 화면

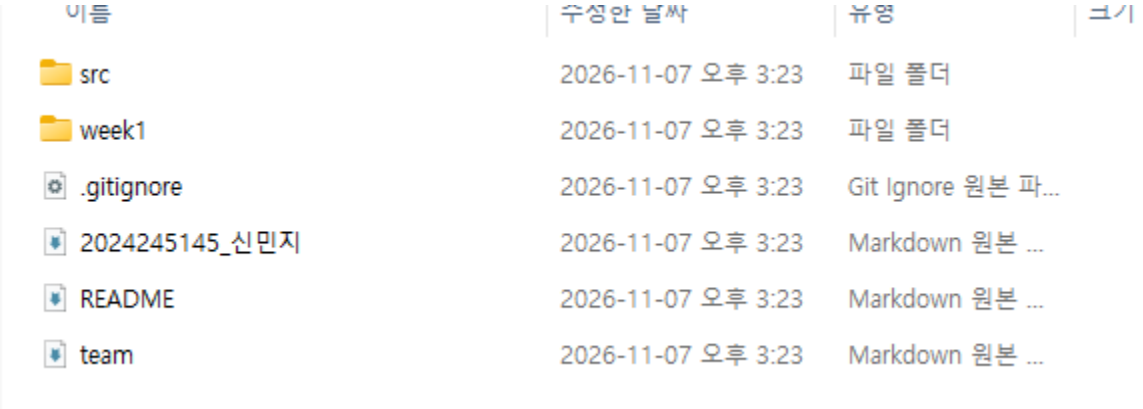


그림 2. ZIP 으로 내려받아 압축을 푼 폴더 (한글 파일명 정상)

2. 프로젝트 배경지식

2.1 경로 계획 문제에서 샘플링이 필요한 이유와 해당 방법

경로 계획은 설정 공간(C-space)에서 장애물과 부딪히지 않는 영역(Q_free) 안에서 출발 자세부터 목표 자세까지 이어지는 길을 찾는 문제이다. 그런데 로봇의 차원, 즉 자세를 나타내는 값의 개수가 높아지면 Q_free 전체를 미리 계산하거나 그리는 것이 불가능해진다. 관절이 2 개인 팔은 설정 공간을 평면에 그려 볼 수 있지만, 관절이 5 개만 되어도 장애물이 설정 공간의 어느 부분을 차지하는지 직접 구하기 어렵다.

그래서 설정 공간에 점(자세)을 무작위로 뿌리고, 각 점을 키네마틱스로 계산해 작업 공간에서 장애물과 충돌하는지 하나씩 검사한 뒤, 충돌하지 않는 안전한 점만 남겨 연결하는 샘플링 방법을 이용한다.

지금까지 학습한 방법 중 샘플링 기반 방법은 다음 세 가지이다.

방법	동작 방식
PRM (Probabilistic Roadmap)	점을 넓게 뿌려 지도(로드맵)를 먼저 만들고, 그 지도 위에서 길을 찾는다
RRT (Rapidly-exploring Random Tree)	출발점에서 나무처럼 가지를 뻗어 나가며 목표를 찾는다
EST (Expansive-Space Tree)	나무를 키울 때 각 노드 주변의 샘플 밀도를 고려하여, 주변에 노드가 적은(덜 탐색된) 영역의

방법	동작 방식
	노드가 확장될 확률을 높여 아직 탐색되지 않은 공간으로 뻗어 나간다

다만 샘플링 방법이 항상 좋은 것은 아니다. 길이 있더라도 통로가 좁으면 그 안에 점이 잘 떨어지지 않아 길이 없는 것처럼 보이는 좁은 통로 문제(narrow passage problem)가 있고, 찾은 경로가 가장 짧은 길이라는 보장도 없다.

2.2 칼만 필터(KF)란 무엇이고, 언제 필요한가

칼만 필터는 베이즈 필터의 원리를 기반으로 로봇의 위치나 속도 같은 상태를 확률적으로 추정하는 방법이다. 베이즈 필터는 상태의 확률분포를 추정하는 일반적인 방법이지만, 격자 기반으로 구현하면 공간이 커질수록 모든 격자의 확률을 저장해야 하므로 메모리 부담이 커진다.

칼만 필터는 선형 시스템과 가우시안 잡음이라는 가정 아래, 확률분포를 가우시안 분포 하나로 표현하고 평균과 공분산만을 이용해 효율적으로 상태를 추정한다. 또한 예측(prediction)과 업데이트(update)를 반복하며, 칼만 게인을 이용해 예측값과 센서 측정값을 결합한다. 이동하면 잡음 때문에 분포가 넓게 퍼지고, 센서로 측정하면 분포가 다시 좁아진다.

칼만 필터는 로봇이 자신의 정확한 위치를 모르고, 이동하면서 계산한 위치와 센서가 알려주는 위치에도 각각 오차가 있을 때 사용한다. 예를 들어 이번 프로젝트처럼 GPS 로 캠퍼스에서 내 위치를 찾는 경우, GPS 측정값은 수 미터씩 흔들리고 이동 모델로 계산한 위치는 오차가 점점 쌓인다. 칼만 필터는 이동 모델과 GPS 측정값 각각의 불확실성을 고려하여 두 정보를 결합함으로써 위치 추정 오차를 줄이는 데 활용할 수 있다.

2.3 파티클 필터란 무엇이고, 언제 필요한가

파티클 필터는 여러 개의 파티클(점)로 로봇의 상태에 대한 belief 를 표현하고, 예측 → 가중치 계산 → 리샘플링을 반복하여 위치를 추정하는 샘플링 기반 베이즈 필터이다. 각 파티클은 로봇이 있을 수 있는 하나의 후보 위치이며, 파티클이 많이 모인 곳일수록 로봇이 있을 가능성이 높다.

단계	하는 일
예측	로봇이 움직인 만큼 모든 파티클을 잡음을 섞어 이동시킨다
가중치 계산	센서 측정값과 잘 맞는 파티클에 높은 가중치를 준다

단계	하는 일
리샘플링	가중치가 높은 파티클은 늘리고, 낮은 파티클은 버린다

칼만 필터가 belief 를 가우시안 분포 하나(평균과 공분산)로 표현하는 parametric 방법이라면, 파티클 필터는 분포의 모양을 미리 정하지 않는 non-parametric 방법이다. 따라서 파티클 필터는 기본 칼만 필터와 달리 선형성이나 가우시안 분포를 가정할 필요가 없어 비선형·비가우시안 문제에도 적용할 수 있고, 봉우리가 여러 개인 분포도 표현할 수 있다. 다만 파티클을 예측하고 평가하기 위한 이동 모델과 센서 모델은 여전히 필요하다.

파티클 필터는 로봇이 자신의 위치를 정확히 모르고, 가능한 위치가 여러 군데일 때 필요하다. 칼만 필터는 봉우리가 하나뿐이라 후보가 여러 개인 상황을 표현할 수 없지만, 파티클 필터는 여러 후보를 동시에 유지하다가 센서 정보가 쌓이면서 하나로 좁혀 갈 수 있다. 예를 들어 여러 버스 노선이 겹치는 구간에서는 내가 어느 버스에 타고 있는지 후보가 여러 개이지만, 커브를 한 번 돌면 그 길로 가는 노선만 남아 후보가 크게 줄어든다.

2.4 Importance sampling 이란 무엇이고, 파티클 필터에서 왜 필요한가

Importance sampling(중요도 샘플링)은 목표 분포에서 직접 샘플링하기 어려울 때, 샘플링하기 쉬운 제안 분포에서 샘플을 뽑고 중요도 가중치를 부여하여 목표 분포를 추정하는 방법이다.

파티클 필터에서 목표 분포는 이동 정보와 센서 관측을 모두 반영한 로봇 상태의 사후 확률분포(belief)이며, 기본적인 방식에서 제안 분포는 이동 모델로 예측한 분포이다. 사후 확률분포에서는 직접 파티클을 뽑을 수 없으므로, 먼저 이동 모델로 예측한 파티클을 만들고 각 파티클에 센서 측정값과 잘 맞는 정도를 반영한 가중치를 부여한 뒤, 이 가중치를 이용해 리샘플링한다.

파티클 필터는 여러 위치 후보를 유지하는 방법이고, importance sampling 은 그 후보들에 적절한 가중치를 부여해 원하는 확률분포를 추정하도록 돕는 방법이다. 이 가중치가 없으면 센서 정보가 반영되지 않아 파티클이 이동할 때마다 퍼지기만 하고 센서 관측과 맞는 위치 근처로 모이지 않는다. 따라서 importance sampling 은 파티클 필터가 맞는 후보는 남기고 틀린 후보는 버리며 실제 위치를 찾아가게 해 주는 핵심 단계이다.

2.5 Localization 문제를 푸는 이유와 현실의 대응 문제

Localization 은 로봇이 지도 안에서 자신의 위치가 어디인지 추정하는 문제로, 무인 자율주행에서 가장 기본적인 기술이다. 예를 들어 청소로봇이 거실에 있다고 생각해 보자. 청소로봇이 충전기로 돌아가려면 충전기의 위치뿐 아니라 자신이 지금 어디에 있는지도 알아야 한다. Localization 이 "나는 지금 어디에 있는가?"의 문제라면, Path Planning 은 "현재 위치에서 목표까지 어떤 길로 갈 것인가?"의 문제이다. 경로 계획을 잘하더라도 로봇이 자신의 위치를 잘못 알고 있다면 계획한 경로를 제대로 따라가기 어렵다. 이것이 Localization 문제를 푸는 이유이다.

로봇이 자신의 위치를 정확하게 알 수 없는 이유는 다음과 같다. 예를 들어 로봇이 1m 이동하라는 명령을 받았을 때, 바퀴가 미끄러져 실제로는 0.9m 만 이동했는데 로봇은 1m 이동했다고 생각할 수 있다. 이런 이동 오차는 계속 쌓이고, GPS 나 카메라 같은 센서에도 잡음이 있기 때문에 로봇은 위치를 확률적으로 추정해야 한다. 위치를 추정하는 방법으로는 이동 정보와 센서 정보를 이용해 위치 확률을 갱신하는 베이즈 필터, 평균과 공분산으로 상태를 추정하는 칼만 필터, 여러 위치 후보에 가중치를 부여해 추정하는 파티클 필터가 있다.

현실 문제	Localization 이 필요한 이유
청소로봇	배터리가 떨어지면 충전기로 돌아가야 한다
자율주행차	도로 위에서 자신의 위치를 정확히 확인해야 한다
물류창고 로봇	정확한 선반 위치까지 이동해야 한다
이번 프로젝트	GPS 와 필터를 이용해 캠퍼스에서 내 위치를 찾는다

특히 Kidnapped Robot Problem(로봇 납치 문제)도 중요하다. 누군가 로봇을 갑자기 다른 장소로 옮겨 놓으면 기존의 위치 추정이 틀어지므로, 로봇은 센서 정보를 이용해 자신의 위치를 다시 찾아야 한다.

2.6 Occupancy grid map 이란 무엇이고, 센서 측정으로 만드는 방법

알고리즘을 적용하려면 먼저 데이터를 어떻게 표현할지 정해야 하며, 지도를 표현하는 방법에는 격자(grid) 기반, 특징점(feature, 랜드마크) 기반, 위상(topological, 그래프) 기반 방법이 있다. Occupancy Grid Map(점유 격자 지도)은 이 중 격자 기반 방법으로, 공간을 작은 격자 셀로 나누고 각 셀에 장애물이 존재할 확률을 저장하는 지도 표현 방법이다. 각 셀의 값은 0 에서 1 사이의 확률로,

1 에 가까우면 장애물이 있는 셀(occupied), 0 에 가까우면 비어 있는 셀(free)을 의미한다. 아직 측정하지 않은 셀은 0.5(unknown)로 시작한다.

LiDAR 와 같은 거리 센서로 측정할 때, 센서 빔이 지나간 셀은 비어 있을 가능성이 높으므로 점유 확률을 낮추고, 빔이 장애물에 닿은 셀은 점유 확률을 높인다. 센서에는 잡음이 존재하므로 한 번의 측정으로 확정하지 않고, 로봇이 이동하며 여러 번 측정할 때마다 베이즈 방식으로 각 셀의 점유 확률을 갱신한다. 일관된 센서 측정이 누적되면 점유된 셀은 점유 확률이 높아지고 비어 있는 셀은 점유 확률이 낮아져, 지도에 대한 확신이 점차 높아진다. 반면 센서가 닿지 않은 셀은 0.5(unknown)로 남는다.

2.5 의 Localization 이 지도가 주어졌을 때 로봇의 위치를 추정하는 문제라면, Occupancy Grid Map 을 만드는 것은 로봇의 위치를 알 때 지도를 만드는 문제이다. 이 두 가지를 동시에 수행하는 것이 SLAM(Simultaneous Localization and Mapping)이다.